

Módulo 4: Aprendizaje Automático

Clase 3: Regresión Lineal Simple y Múltiple

Diplomado en Ciencia de Datos

4 de julio de 2026

¿Qué es la Regresión Lineal?

Definición

Es un algoritmo fundamental de Aprendizaje Supervisado utilizado para predecir o estimar un valor numérico continuo (como precios, temperatura o ventas), basándose en la relación matemática entre distintas variables.

Nuestro Caso de Estudio: Seguros Médicos

Deseamos estimar el costo anual del seguro médico de un paciente. ¿De qué factores estadísticos depende esta estimación?

- **Edad:** Variable Numérica Continua.
- **Índice de Masa Corporal (IMC):** Variable Numérica Continua.
- **Condición de Fumador:** Variable Categórica.

Paso 1: Preprocesamiento de Datos

Antes de aplicar cualquier modelo de regresión, la matriz de datos debe estar estructurada matemáticamente. Recordando la Sesión 2, aplicaremos tres procesos críticos:

- 1 **Imputación de Valores Nulos:** Reemplazo de datos faltantes.
- 2 **Codificación Categórica (One-Hot Encoding):** Transformación de texto a binario.
- 3 **Escalado de Magnitudes:** Estandarización de variables numéricas.

Comandos en Pandas: Imputación y Codificación

En nuestro cuaderno de trabajo (Jupyter Notebook), utilizaremos los siguientes comandos de la librería pandas:

1. Imputación mediante Mediana

```
mediana_bmi = df_seguros['bmi'].median()  
df_seguros['bmi'] = df_seguros['bmi'].fillna(mediana_bmi)
```

El método `fillna()` busca cualquier valor NaN en la columna y lo reemplaza por el valor estadístico de la mediana.

2. One-Hot Encoding

```
df_final = pd.get_dummies(df_seguros, columns=['sex', 'smoker'],  
                           drop_first=True)
```

El comando `get_dummies()` convierte variables categóricas en columnas binarias (0 y 1). El parámetro `drop_first=True` es esencial en regresión lineal para evitar la redundancia matemática (colinealidad perfecta).

La regresión asume que existe una **relación lineal** entre la variable independiente (X) y la objetivo (y).

Para verificar esto, utilizamos el **Coeficiente de Correlación de Pearson** (r).

- $r = 1,0$: Relación positiva perfecta.
- $r = -1,0$: Relación negativa perfecta.
- $r = 0,0$: Ausencia de correlación lineal.

Comandos: Visualización de Dispersión y Correlación

Podemos verificar la correlación tanto matemática como visualmente:

Gráfico de Dispersión (Scatter Plot)

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(data=df_final, x='age', y='charges')
plt.show()
```

Esto genera un plano cartesiano donde cada punto es un paciente. El eje X representa la edad y el eje Y los cargos.

Cálculo Estadístico

```
correlacion = df_final['age'].corr(df_final['charges'])
```

El método `corr()` de Pandas retorna el valor exacto de Pearson.

Regresión Lineal Simple: Teoría

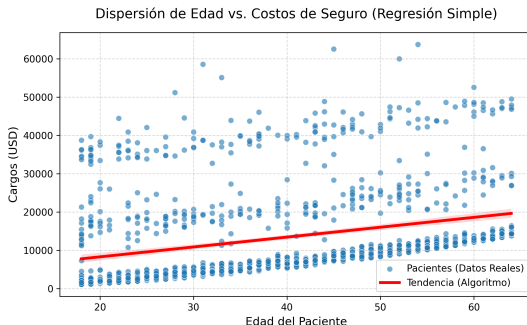
En su forma más elemental, intentamos estimar el resultado usando **una sola variable predictora**.

La Ecuación de la Línea Recta:

$$y = \beta_0 + \beta_1 X$$

- y (**Target**): El valor que estimamos (Cargos Médicos).
- X (**Feature**): La variable independiente (Edad).
- β_1 (**Coeficiente**): La pendiente. Cuánto incrementa el costo por cada año de edad.
- β_0 (**Intercepto**): El valor base si X fuera cero.

Guía Visual: Regresión Simple



Interpretación:

- **Puntos Azules (Realidad):** Cada paciente real de nuestra base de datos.
- **Línea Roja (Tendencia):** El "estimador" matemático calculando la mejor trayectoria media.
- **Conclusión:** Al haber tanta dispersión alejada de la línea roja, deducimos empíricamente que un solo factor (Edad) no es suficiente.

Comandos Scikit-Learn: Regresión Simple (1/2)

El flujo de trabajo en la librería `scikit-learn` consta de instanciar el modelo, ajustarlo a los datos (Entrenamiento) y generar predicciones.

División de Entrenamiento y Prueba

```
from sklearn.model_selection import train_test_split

X = df_final[['age']] # Matriz 2D
y = df_final['charges'] # Vector 1D
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
```

Separamos el 80 % de los datos para enseñar al modelo, y ocultamos el 20 % (test) para evaluar su precisión de forma imparcial.

Entrenamiento (Fit) y Predicción (Predict)

```
from sklearn.linear_model import LinearRegression

# 1. Crear el algoritmo
modelo_simple = LinearRegression()

# 2. Entrenar (El algoritmo calcula los betas)
modelo_simple.fit(X_train, y_train)

# 3. Predecir datos nuevos
predicciones = modelo_simple.predict(X_test)
```

El método `fit()` es donde ocurre el cálculo de Mínimos Cuadrados Ordinarios (OLS). El método `predict()` aplica la ecuación generada a nuevos datos.

Evaluación de Desempeño: ¿Es un buen modelo?

Para auditar la validez del modelo, comparamos los valores reales (y_{test}) contra nuestras proyecciones (predicciones).

- **MAE (Error Absoluto Medio):** Promedio del error en la misma escala (ej. Dólares).
- **MSE (Error Cuadrático Medio):** Penaliza severamente los errores grandes elevándolos al cuadrado.
- **RMSE (Raíz del MSE):** Devuelve el MSE a la unidad original para su interpretación de negocio.
- **R^2 (Coeficiente de Determinación):** Representa el porcentaje (0.0 a 1.0) de la varianza en y que es explicada por el modelo. Un valor de 0,80 significa un modelo 80 % certero.

Extracción de Métricas de Evaluación

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# R-Cuadrado (0.0 a 1.0)
r2 = r2_score(y_test, predicciones)

# RMSE (En dolares)
mse = mean_squared_error(y_test, predicciones)
rmse = np.sqrt(mse)

print(f"R-Cuadrado: {r2}")
print(f"Error RMSE: {rmse}")
```

Regresión Lineal Múltiple: La Realidad Multidimensional

Un solo factor (como la Edad) genera modelos imprecisos. La regresión **múltiple** incorpora varias variables, formando un "hiperplano" multidimensional:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \dots$$

- X_1 : Edad del paciente
- X_2 : Índice de Masa Corporal (IMC)
- X_3 : Estado de Fumador (0 = No, 1 = Sí)

El código de Scikit-Learn es **exactamente el mismo**. La única diferencia es que la matriz X contendrá múltiples columnas.

Escalado en la Regresión Múltiple

Al tener múltiples variables, sus escalas numéricas varían (ej. Edades en decenas, pero salarios en miles). Debemos homogeneizarlas para que el algoritmo no asuma erróneamente que los números grandes son más importantes.

StandardScaler

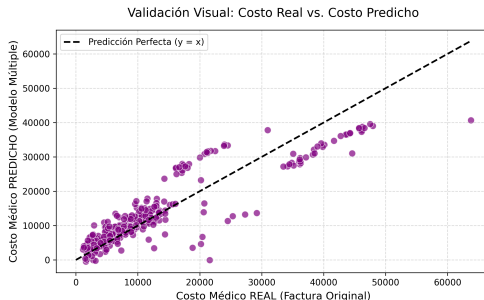
```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_mult[['age', 'bmi']] = scaler.fit_transform(X_mult[['age', 'bmi']])
```

El StandardScaler transforma las columnas continuas para que tengan media 0 y desviación estándar 1 (Z-Score).

Guía Visual: Realidad vs. Predicción

No podemos dibujar un hiperplano en 4D o 5D. Para evaluar un modelo múltiple, hacemos que compita contra la realidad:



Interpretación:

- **Eje X:** Cargos Reales. **Eje Y:** Cargos Predichos.
- **Línea Negra (Diagonal):** La Perfección matemática ($y = x$).
- Si el modelo es preciso, los puntos morados abrazarán ajustadamente la línea diagonal.

Inferencia de Negocio: Interpretando los Coeficientes

La Regresión Lineal destaca por su "transparencia". Podemos extraer los pesos algorítmicos (β) asociados a cada variable.

Extracción de Coeficientes

```
# Imprimir coeficientes asignados a las variables  
print(modelo_multiple.coef_)
```

Variable Evaluada	Impacto Económico Monetario (β)
Edad (Estandarizada)	+\$3,500
IMC (Estandarizada)	+\$2,000
Fumador (smoker_yes)	+\$23,800

Análisis Analítico: Si comparamos a dos pacientes idénticos en edad y peso físico, el riesgo de ser **Fumador** eleva el costo médico predicho en casi \$24,000 USD adicionales.

- **Preprocesamiento Crítico:** Transformar el texto (Fumador: Sí/No) a números (One-Hot Encoding) y escalar las magnitudes es un requisito ineludible.
- **Flujo Estándar:** Todo modelo en Python sigue la secuencia Instanciar $\rightarrow$ `fit()` $\rightarrow$ `predict()`.
- **Simple vs Múltiple:** Agregar dimensiones incrementa dramáticamente la exactitud predictiva (ej. El R^2 subió del 9 % al 76 %).
- **Inferencia:** Los algoritmos lineales permiten extraer reglas de negocio directas mediante sus coeficientes.